ENTERPRISE TECHNICAL ARCHITECTURE DOCUMENT

SIMPLEX Central Operations Console

Architecture, Codebase, Security & Database Specification

Platform: SIMPLEX v1.0
Release Date: September 16, 2026
Classification: Confidential & Proprietary

SIMPLEX Central Operations Console

System Architecture, Codebase & Database Specification

Executive Summary

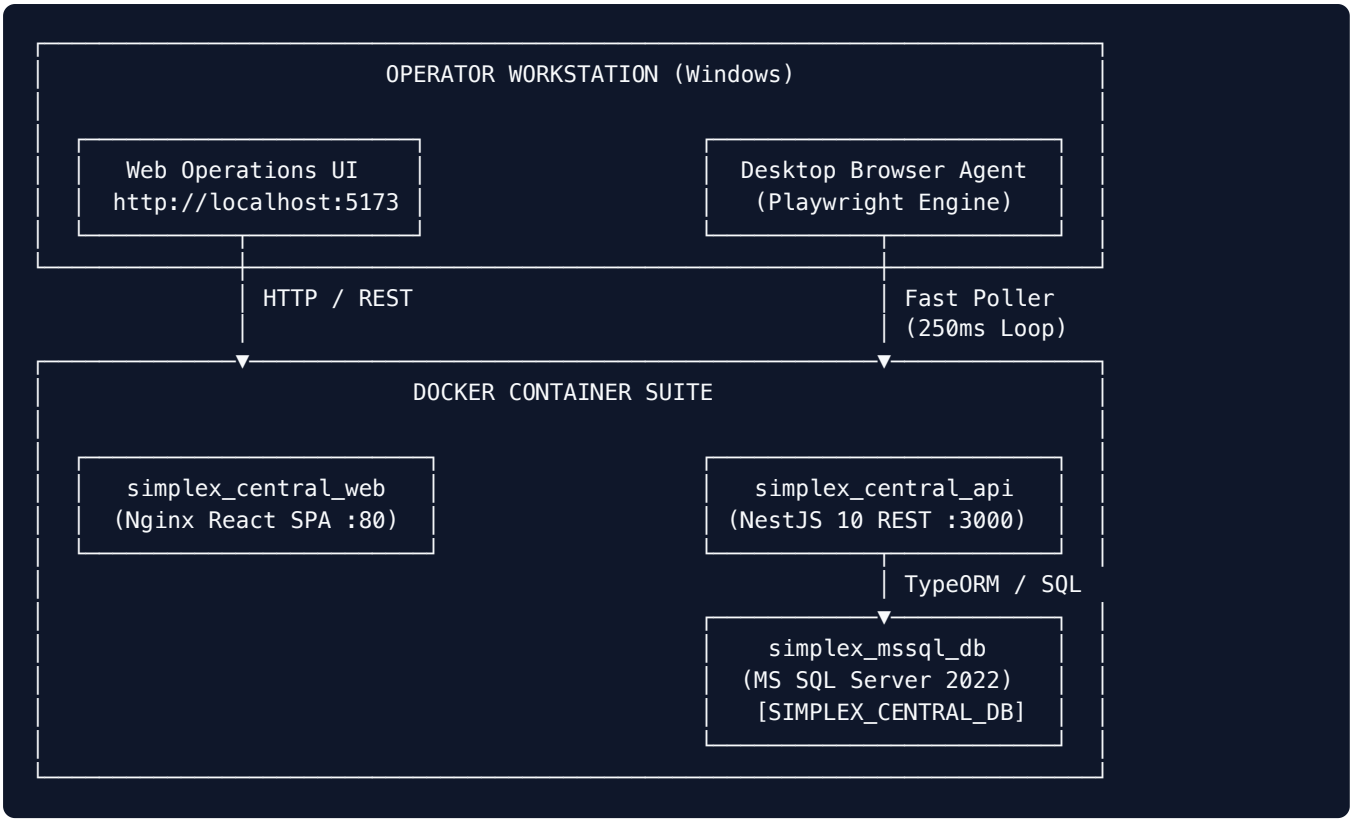
The **SIMPLEX Central Operations Console** is an enterprise-grade multi-client management, role-based access control (RBAC), and automated desktop browser orchestration platform designed for healthcare, clinic, and hospital networks.

The system empowers central administrators and operators to securely monitor and orchestrate remote client instances (e.g., `https://staging.simplexworld.com/MasterV9.3`), execute bulk user and medical resource imports, perform real-time user discovery/synchronization, automate user password resets, and launch interactive browser sessions with single-click automated authentication—**without ever exposing plaintext client credentials to operators or client-side storage**.

---

1. High-Level Architecture & Component Topography

The platform is architected around a decoupled, 4-tier hybrid architecture combining isolated Docker container services with an on-host desktop automation daemon:



Component Roles & Communication

1. **Web Operations Console (`simplex_central_web`)**:
   - Single Page Application (SPA) serving central operators.
   - Communicates exclusively with the Central API via secure JSON REST endpoints (`/api/v1`).
   - Maintains operator session through short-lived JWT access tokens and HTTP-only refresh tokens.
2. **Central REST API (`simplex_central_api`)**:
   - Core business logic, RBAC security guards, envelope encryption, and task dispatching queue.
   - Interacts with MS SQL Server using TypeORM over persistent connection pools.
   - Manages asynchronous automation run leases, agent pairing, and telemetry ingest.
3. **Database Server (`simplex_mssql_db`)**:
   - Dedicated Microsoft SQL Server 2022 instance containing `SIMPLEX_CENTRAL_DB`.
   - Hardened with a sandboxed service account (`simplex_app_user`) explicitly forbidden from accessing any other databases on the instance (`DENY VIEW ANY DATABASE`).
4. **Desktop Browser Automation Agent (`simplex-browser-agent`)**:
   - Executes natively on the Windows host machine to interact with physical desktop display drivers.
   - Leverages Microsoft Playwright (Chromium) to launch headed and headless browser sessions.
   - Authenticates with the Central API using a cryptographic shared secret (`AGENT_SHARED_SECRET`).

---

2. Codebase & Technology Stack Details

The codebase is organized as a high-performance TypeScript monorepo structured into applications and shared packages:

```
SIMPLEX/
├── apps/
│   ├── web/                    # React 18 Frontend Console
│   ├── api/                    # NestJS 10 Backend API Service
│   └── desktop-agent/          # Native Desktop Automation Runner
├── packages/
│   ├── shared/                 # Zod Schemas, DTOs & URL Resolvers
│   ├── database/               # TypeORM Entities, Migrations & Crypto
│   └── automation/             # Playwright Engine & Workflow Executors
└── delivery/                   # Production Deployment ZIP & Scripts
```

A. Frontend Application (`apps/web`)

- **Technology**: React 18, TypeScript, Vite 5, Tailwind CSS, Lucide Icons.
- **Key Modules**:
  - `src/pages/ClientsPage.tsx`: Client registration, live connectivity status, automated sync triggers, and instant interactive session launch.
  - `src/pages/UsersPage.tsx`: Synchronized client user table, remote status indicators, live filtering, and ephemeral password reset modal.
  - `src/pages/ResourcesPage.tsx`: Multi-stage resource onboarding wizard (Resource -> User -> Role -> Registration Mapping -> EMR Form Transfer).
  - `src/pages/ImportsPage.tsx`: Bulk Excel and CSV upload, schema preview, and staged batch validation.
  - `src/pages/AuditLogsPage.tsx`: Filterable security audit logs showing operator action trails.
- **Production Delivery**: Pre-built static bundles packaged into an optimized Alpine Nginx image (`ghcr.io/rajeshsupport/simplex/simplex-web:latest`).

B. Backend API Application (`apps/api`)

- **Technology**: NestJS 10, Express, TypeORM, Passport JWT, Argon2, Crypto.
- **Key Modules & Services**:
  - `AuthModule`: Operator authentication, Argon2id password verification, JWT issuance and refresh rotation.
  - `ClientsModule`: Client registration, dynamic route sanitization, credential encryption via AES-256-GCM.
  - `ClientUsersModule`: Ingests scraped user snapshots, manages live option inspection, and coordinates remote password resets.
  - `ClientResourcesModule`: Orchestrates multi-step medical provider onboarding and form assignments.
  - `AgentsModule`: Handles runner pairing, lease allocation, heartbeat health checks, and task cancellation.
  - `AuditModule`: Centralized audit logging capturing actor, IP, timestamp, entity, and correlation ID.
- **Production Delivery**: Pre-compiled standalone Docker container image (`ghcr.io/rajeshsupport/simplex/simplex-api:latest`). Zero source code is exposed.

C. Desktop Browser Automation Agent (`apps/desktop-agent` & `packages/automation`)

- **Technology**: Node.js v20+, Microsoft Playwright, Chromium.
- **Key Engines**:
  - `cli-runner.js`: Fast 250ms polling loop retrieving queued tasks and reporting real-time telemetry.
  - `automation-worker.js`: Dispatches tasks to specialized automation handlers.
  - `UserManagementExecutor`:
    - **Open Interactive Session ("Open & Login")**: Spawns visible headed browser, injects credentials, logs in, and transfers session to human operator.

        - **Headless User Directory Sync**: Automatically scrapes paginated user tables in chunks of 100 users.
        - **Live Password Reset**: Navigates to target user row, executes reset action, captures confirmation dialogs, and applies fallback defaults (`Simplex@123`).
    - `ResourceManagementExecutor` : Executes sequential stages:
        1. Create Resource (`/addResourceParentDetails`)
        2. Create Associated User (`/addUsers`)
        3. Assign Roles (`/addUserRole`)
        4. Map User to Resource with Registration (`/ResourceParent`)
        5. Transfer EMR Forms to Target Branch (`/emrPanelSelection`)

D. Shared Libraries (`packages/`)

- `@hmc/shared` : Zod validation schemas (`CreateClientSchema` , `ImportJobSchema`), TypeScript interfaces, and resilient URL resolvers (`resolveClientRoleUrl` , `resolveClientRoute`).
- `@hmc/database` : TypeORM entity definitions, migrations, cryptographic envelope encryption helpers, and data-source configuration.
- `@hmc/automation` : Playwright browser lifecycle manager, profile isolation, and dynamic DOM selector resolvers.

---

3. Comprehensive Database Details (`SIMPLEX_CENTRAL_DB`)

The database runs on **Microsoft SQL Server 2022**. It enforces strict isolation, relational integrity, indexing, and tamper-evident audit trails.

Database Connection Parameters:

- **Server / Container**: `simplex_mssql_db`
- **Internal Port**: `1433` (External host port: `14333`)
- **Database Name**: `SIMPLEX_CENTRAL_DB`
- **Application User**: `simplex_app_user` (db_owner on `SIMPLEX_CENTRAL_DB` , DENY VIEW ANY DATABASE on `master`)
- **Master Admin User**: `sa` (restricted to deployment and maintenance tasks)

Complete Database Schema & Table Catalog

The database contains 23 core tables organized across 6 functional domains:

Domain 1: Authentication & Access Control (RBAC)

1. `application_users` :
    - Stores Central Console operator accounts.
    - Columns: `id` (UUID PK), `username` (VARCHAR 50 UNIQUE), `email` (VARCHAR 255), `fullName` (VARCHAR 150), `passwordHash` (Argon2id), `status` (ACTIVE/INACTIVE), `lastLoginAt` (DATETIME2), `createdAt` , `updatedAt` .
2. `roles` :
    - System access roles (`SUPER_ADMIN` , `OPERATOR` , `AUDITOR`).
    - Columns: `id` (UUID PK), `name` (VARCHAR 50 UNIQUE), `description` , `isSystem` (BIT).
3. `permissions` :

- Granular permission registry (`client.create` , `client.view` , `user.sync` , `user.reset_password` , `resource.manage` , etc.).
- Columns: `id` (UUID PK), `name` (VARCHAR 100 UNIQUE), `category` , `description` .

4. **`user_roles`** :
   - Many-to-many junction table linking operators to roles.
   - Columns: `user_id` (FK to `application_users`), `role_id` (FK to `roles`).
5. **`role_permissions`** :
   - Many-to-many junction table mapping granted permissions to roles.
   - Columns: `role_id` (FK to `roles`), `permission_id` (FK to `permissions`).
6. **`application_login_history`** :
   - Security tracking of all Console login attempts.
   - Columns: `id` (UUID PK), `userId` (FK), `timestamp` (DATETIME2), `ipAddress` , `userAgent` , `status` (SUCCESS/FAILURE), `failureReason` .

Domain 2: Client Instance Management

7. **`clients`** :
   - Registry of connected hospital/clinic endpoints.
   - Columns: `id` (UUID PK), `clientCode` (VARCHAR 50 UNIQUE), `clientName` (VARCHAR 150), `baseUrl` (VARCHAR 500), `applicationPath` (VARCHAR 255), `environment` (Production/Staging/UAT/Development), `applicationVersion` , `loginRoute` , `usersRoute` , `servicesRoute` , `userRoleRoute` , `quickResourceRoute` , `resourceUserRoute` , `status` , `createdAt` , `updatedAt` .
8. **`client_credentials`** :
   - Encrypted login credentials for client portals.
   - Columns: `id` (UUID PK), `clientId` (FK UNIQUE), `credentialName` , `encryptedUsername` (NVARCHAR 500), `usernameIv` , `usernameTag` , `usernameMasked` , `encryptedPassword` (NVARCHAR 500), `passwordIv` , `passwordTag` , `keyVersion` (INT), `isActive` (BIT), `lastTestedAt` (DATETIME2).
9. **`user_client_access`** :
   - Scopes which specific clients an operator is authorized to access.
   - Columns: `id` (UUID PK), `userId` (FK), `clientId` (FK), `accessLevel` (ADMIN/OPERATOR/READONLY).

Domain 3: User Discovery & Synchronization

10. **`client_user_snapshots`** :
    - Complete synchronized user directory discovered from remote client instances.
    - Columns: `id` (UUID PK), `clientId` (FK), `clientCode` , `remoteUserId` , `username` (VARCHAR 100), `firstName` , `middleName` , `lastName` , `fullName` , `nickName` , `email` , `mobileNumber` , `nationality` , `role` (NVARCHAR MAX), `profileRole` , `status` (ACTIVE/INACTIVE), `barcodeNumber` , `hasSignature` (BIT), `hasStamp` (BIT), `hasProfileImage` (BIT), `isPresentRemotely` (BIT), `syncRunId` , `lastSyncedAt` (DATETIME2), `lastVerifiedAt` (DATETIME2), `createdAt` , `updatedAt` .
    - Constraints: Unique index on `(clientId, username)` .

Domain 4: Medical Resource Management & Workflow Automation

11. **`client_resource_snapshots`** :
    - Discovered and onboarded medical providers, rooms, and equipment.

- Columns: `id` (UUID PK), `clientId` (FK), `clientCode`, `remoteResourceId`, `resourceCode`, `resourceName`, `isResourceHuman` (BIT), `remoteResourceTypeId`, `resourceTypeName`, `resourceType`, `remoteSpecialtyId`, `specialtyName`, `department`, `specialization`, `colorIdentificationCode`, `operatingFrom`, `operatingTo`, `selectAllDepartments` (BIT), `selectAllServices` (BIT), `linkedRemoteUserId`, `linkedUsername`, `isShownInRegistration` (BIT), `branchId`, `branchName`, `status`, `isPresentRemotely` (BIT), `workflowStage`, `eclaimStatus`, `eclaimConfigJson`, `emrFormsJson`, `transferConfigJson`, `retryResumeState`, `lastVerifiedAt`, `lastSyncedAt`.
- Constraints: Unique index on `(clientId, remoteResourceId)`.

12. **`client_resource_departments`**:
    - Department mappings assigned to each medical resource.
    - Columns: `id` (UUID PK), `resourceId` (FK), `departmentCode`, `departmentName`.

13. **`client_resource_services`**:
    - Service and billing codes linked to each medical resource.
    - Columns: `id` (UUID PK), `resourceId` (FK), `serviceCode`, `serviceName`.

14. **`resource_import_jobs`**:
    - Header tracking for bulk resource onboarding spreadsheet jobs.
    - Columns: `id` (UUID PK), `clientId` (FK), `clientCode`, `fileName`, `status`, `totalRows`, `completedRows`, `failedRows`, `skippedRows`, `createdBy`, `createdAt`, `updatedAt`.

15. **`resource_import_rows`**:
    - Individual row status and payload for resource spreadsheet imports.
    - Columns: `id` (UUID PK), `jobId` (FK), `rowNumber`, `resourceName`, `stage`, `status`, `retryStartingPoint`, `safeErrorCode`, `safeErrorMessage`, `rawRowJson`.

Domain 5: Batch File Imports

16. **`import_jobs`**:
    - Header tracking for bulk user Excel/CSV import jobs.
    - Columns: `id` (UUID PK), `clientId` (FK), `clientCode`, `fileName`, `status`, `totalRows`, `processedRows`, `successfulRows`, `failedRows`, `createdAt`, `updatedAt`.

17. **`import_job_rows`**:
    - Row-level parsed data and execution results for user imports.
    - Columns: `id` (UUID PK), `importJobId` (FK), `rowIndex`, `rawDataJson` (NVARCHAR MAX), `mappedDataJson`, `status`, `errorMessage`, `resultJson`, `retryCount`.

Domain 6: Browser Automation Orchestration & System Governance

18. **`desktop_agents`**:
    - Registered desktop browser automation runners.
    - Columns: `id` (UUID PK), `agentName`, `machineHostname`, `osInfo`, `assignedUserId` (FK), `authTokenHash`, `status` (ONLINE/BUSY/OFFLINE), `lastHeartbeatAt` (DATETIME2), `systemMetricsJson`.

19. **`automation_runs`**:
    - Queued and executed automation jobs dispatched to agents.
    - Columns: `id` (UUID PK), `clientId` (FK), `desktopAgentId` (FK), `triggeredByUserId`, `runType` (OPEN_INTERACTIVE_CLIENT_SESSION, SYNC_CLIENT_USERS_HEADLESS, RESET_CLIENT_USER_PASSWORD, PROCESS_USER_FULL_WORKFLOW,

INSPECT_CREATE_FORM_METADATA), `status` (PENDING, CLAIMED, RUNNING, COMPLETED, FAILED, TIMED_OUT), `startedAt`, `completedAt`, `totalDurationMs`, `correlationId`, `parametersJson`, `resultSummaryJson`.

20. **`automation_run_steps`**:
    - Granular step-by-step telemetry, duration, and screenshot paths for each run.
    - Columns: `id` (UUID PK), `automationRunId` (FK), `stepIndex`, `stepName`, `status`, `durationMs`, `screenshotFilePath`, `stepDetailsJson`.
21. **`automation_workflows`** & **`automation_workflow_versions`**:
    - Parametric definitions of automation recipes and multi-step UI sequences.
22. **`audit_logs`**:
    - Tamper-evident, immutable audit trail for all operational activities.
    - Columns: `id` (UUID PK), `timestamp` (DATETIME2), `actorUserId`, `actorUsername`, `clientId`, `clientCode`, `action`, `entityType`, `entityId`, `result` (SUCCESS/FAILED), `ipAddress`, `userAgent`, `correlationId`, `detailsJson`.
23. **`error_logs`** & **`retention_policies`**:
    - Centralized application crash logging and automated record lifecycle retention rules.

---

4. Key Security & Cryptographic Mechanisms

1. **AES-256-GCM Envelope Encryption (`client_credentials`)**:
    - Remote client passwords are encrypted using a 256-bit Master Key (`ENCRYPTION_MASTER_KEY`).
    - Each credential row generates a distinct cryptographic Initialization Vector (`IV`) and Authentication Tag (`tag`). Plaintext passwords never touch disk or logs.
2. **Interactive Session Credential Isolation ("Open & Login")**:
    - The desktop agent decrypts the client credentials in local process memory only for the exact duration of the login action.
    - Playwright types the credentials directly into the client login DOM and immediately clears the variables. The human operator never sees the password.
3. **Database Sandbox Isolation**:
    - The SQL Server login `simplex_app_user` is explicitly bound by `DENY VIEW ANY DATABASE`. It is physically impossible for the application or any SQL injection vulnerability to inspect or access other databases on the host instance.
4. **Tamper-Evident Correlation Tracking**:
    - Every user action generates a unique UUID `correlationId` passed across the Web UI, API, Agent, Database, and Audit Logs, allowing end-to-end incident forensics.

---

5. Deployment & Package Topology

The entire solution is packaged in a self-contained, offline-capable delivery bundle: **`simplex-delivery.zip`** (9.0 MB).

```
simplex-delivery/
├── docker-compose.yml        # Orchestrates Database, API, and Web containers
├── .env                      # Production environment configuration & secrets
├── init-db.sql               # Self-healing database schema, tables & permissions
├── start.bat                 # 1-Click launcher (checks Docker, initializes DB, launches UI)
├── stop.bat                  # Graceful container shutdown
├── update.bat                # 1-Click container update from registry (preserves DB volume)
└── agent/
    ├── start-agent.bat       # Native Desktop Browser Automation Agent launcher
    ├── dist/cli-runner.js    # Compiled automation poller daemon
    └── package.json          # Agent dependencies (Playwright, Chromium)
```

Document compiled for SIMPLEX Central Operations Console • Confidential • September 2026